



Otimização de Rotas de Coleta de Resíduos Urbanos (Grafos Eulerianos)

Diego Medeiros Ponte, João Victor Andrade da Cunha Nascimento, João Marcelo de Souza e Paulo
Roberto Fernandes Holanda

Cenário: EcoUrbano — coleta de resíduos urbanos

Objetivo:

encontrar uma rota que percorra cada rua exatamente uma vez

Modelo:

interseções → vértices

ruas → arestas

rota do caminhão → caminho/ciclo euleriano

Oportunidade de solução

- **Problema operacional:**
rotas manuais podem gerar repetição de ruas,
maior tempo de coleta e maior custo operacional.
- **Oportunidade:**
usar grafos para apoiar o planejamento de rotas urbanas.
- **Valor para a EcoUrbano:**
 - diagnóstico automático da malha
 - identificação se a rota é viável
 - geração de sequência de percurso
 - redução de retrabalho operacional
 - apoio à sustentabilidade

Pipeline da solução

Entrada: grafo ou dígrafo da malha urbana

- 1. Classificar a eulerianidade**
- 2. Definir se existe caminho ou circuito**
- 3. Escolher vértice inicial**
- 4. Executar Hierholzer**
- 5. Validar a rota final**

Determinação da euleridade

```
func Classifica(g *grafo.Grafo) ResultadoEuler {  
    if len(g.Vertices) == 0 {  
        return ResultadoEuler{  
            Classe: CircuitoEuleriano,  
            Conexo: true,  
        }  
    }  
  
    if g.Direcionado {  
        return classificaDigrafo(g)  
    }  
  
    return classificaGrafo(g)  
}
```


Eulerianidade em grafos

```
for _, v := range g.Vertices {  
    if g.GrauVertice(v)%2 != 0 {  
        impares = append(impares, v)  
    }  
}  
  
if len(impares) == 0 {  
    return ResultadoEuler{  
        Classe: CircuitoEuleriano,  
        VerticeInicial: g.Vertices[0],  
    }  
}  
  
if len(impares) == 2 {  
    return ResultadoEuler{  
        Classe: CaminhoEuleriano,  
        GrausImpares: impares,  
        VerticeInicial: impares[0],  
    }  
}
```

Grafo não direcionado:

- 0 vértices ímpares
→ Circuito Euleriano
- 2 vértices ímpares
→ Caminho Euleriano
- Mais de 2 vértices ímpares
→ Não-Euleriano

Eulerianidade em digrafos

```
diff := saida[v] - entrada[v]

if diff != 0 {
    desbalanceados[v] = diff
}

if diff == 1 {
    inicioCount++
    inicio = v
}

if diff == -1 {
    fimCount++
}
```

- Circuito Euleriano
Entrada = saída em todos

- Outro caso
→ Não-Euleriano

- Caminho Euleriano
1 vértice com saída - entrada = +1
1 vértice com saída - entrada = -1

```
if len(desbalanceados) == 0 {
    Classe: CircuitoEuleriano
}

if inicioCount == 1 && fimCount == 1 && len(desbalanceados) == 2 {
    Classe: CaminhoEuleriano
    VerticeInicial: inicio
}
```

Entrada controlada e escolha do vértice inicial

```
res := Classifica(g)
if res.Classe == NaoEuleriano {
    return TrilhaEuler{}, errSemTrilhaEuler
}

inicioEscolhido := escolheInicio(g, inicio, res.VerticeInicial)

if !inicioValidoGrafo(g, inicioEscolhido, res) {
    return TrilhaEuler{}, errInicioInvalido
}

sequencia, err := executarHierholzer(g.Clone(), inicioEscolhido)
```


Como a rota é construída

```
func executarHierholzer(g *grafo.Grafo, inicio string) ([]string, error) {  
    if inicio == "" {  
        return nil, nil  
    }  
  
    atual := inicio  
    pilha := make([]string, 0)  
    reversa := make([]string, 0)  
  
    for len(pilha) > 0 || len(g.ListaAdj[atual]) > 0 {  
        if len(g.ListaAdj[atual]) > 0 {  
            proximo := g.ListaAdj[atual][0]  
            pilha = append(pilha, atual)  
            if !removeUmaAresta(g, atual, proximo) {  
                return nil, fmt.Errorf("remover aresta %s -> %s: %w", atual, proximo,  
errSemTrilhaEuler)  
            }  
            atual = proximo  
            continue  
        }  
  
        reversa = append(reversa, atual)  
        atual = pilha[len(pilha)-1]  
        pilha = pilha[:len(pilha)-1]  
    }  
  
    reversa = append(reversa, atual)  
    inverte(reversa)  
    return reversa, nil  
}
```

Aplicação prática e visão de mercado

Valor operacional da solução

Para uma empresa de coleta:

- reduz repetição de ruas
- economiza combustível
- diminui tempo de rota
- reduz emissão de CO₂
- facilita geração de folha de rota

O sistema entrega:

- diagnóstico da malha
- indicação se a rota é viável
- sequência de percurso quando existe rota euleriana
-

Limite:

não otimiza trânsito, distância ou peso;
garante cobertura euleriana da malha.

Além da coleta de lixo:

- entrega de correspondências
- inspeção de redes elétricas ou fibra óptica
- manutenção de tubulações
- leitura de medidores
- rotas de drones para inspeção
- planejamento de rondas de segurança